

CHAPTER 6: SOFTWARE QUALITY ASSURANCE

6.1 CONCEPTS, SOFTWARE QUALITY ASSURANCE

❖ INTRODUCTION TO QUALITY

Quality is a complex and multifaceted concept that can be described from five different points of view:

- **Transcendental view:** something that immediately recognizes, but cannot explicitly define.
- **User view:** If a product meets an end user's specific goals, it exhibits quality.
- **Manufacturer's view:** Product conforms to the original specification.
- **Product view:** Quality can be tied to inherent characteristics (e.g., functions and features) of a product.
- **Value-based view:** How much a customer is willing to pay for a product?

Thus, Quality can be defined as: Degree of excellence or Fitness for purpose or Best for the customer's use and selling price or Total characteristics of an entity that bear on its ability to satisfy stated or implied needs (ISO).

Quality of design refers to the characteristics that designers specify for a product. The grade of materials, tolerances, and performance specifications all contribute to the quality of design. As higher-grade materials are used, tighter tolerances and greater levels of performance are specified. Quality of conformance focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

❖ SOFTWARE QUALITY

Software quality can be defined as: An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it. The definition serves to emphasize three important points:

- **Effective software process:** establishes the infrastructure that supports any effort at building a high-quality software product.
- **Useful product:** delivers the content, functions, and features that the end user desires in a reliable, error-free way.

- **Added measurable values:** For **producer**: high-quality software requires less maintenance effort, fewer bug fixes, and reduced customer support.
For **end user**:
(1) greater software product revenue,
(2) better profitability when an application supports a business process, and/or
(3) improved availability of information that is crucial or vital for the business.

To achieve high-quality software, four activities must occur: proven software engineering process and practice, solid project management, comprehensive quality control, and the presence of a quality assurance infrastructure.

❖ Quality Factor

The ISO 9126 standard was developed in an attempt to identify the key quality attributes for computer software. The standard identifies six key quality attributes:

1. **Functionality:** The degree to which the software satisfies stated need as indicated by the following subattributes: Suitability, accuracy, interoperability, compliance, and security.

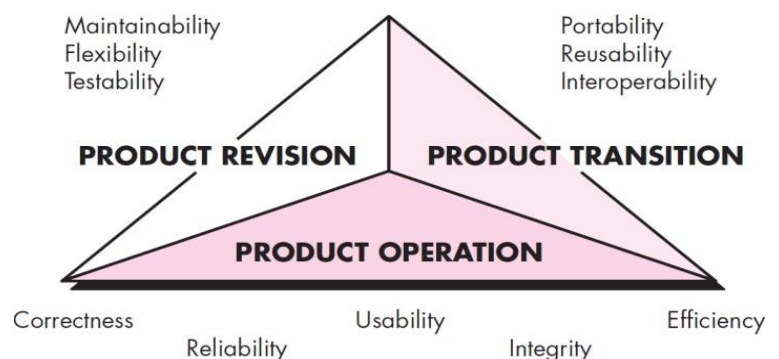


Fig. McCall's software quality factors

2. **Reliability:** The amount of time that the software is available for use as indicated by the following subattributes: maturity, fault tolerance, recoverability.
3. **Usability:** The degree to which the software is easy to use as indicated by the following subattributes: understandability, learnability, operability.
4. **Efficiency:** The degree to which the software makes optimal use of system resources as indicated by the following subattributes: time behavior, resource behavior.
5. **Maintainability:** The ease with which repair may be made to the software as indicated by the following subattributes: analyzability, changeability, stability, testability.
6. **Portability:** The ease with which the software can be transposed from

one environment to another as indicated by the following subattributes: adaptability, installability, conformance, replaceability.

❖ COST OF QUALITY

The software cost depends on *time and money*. The *cost of quality* includes all costs incurred in the pursuit of quality or in performing quality-related activities and the downstream costs of lack of quality. To understand these costs, an organization must collect metrics to provide a baseline for the current cost of quality, identify opportunities for reducing these costs, and provide a normalized basis of comparison. The cost of quality can be divided into costs associated with prevention, appraisal, and failure.

- **Prevention costs** include (1) the cost of management activities required to plan and coordinate all quality control and quality assurance activities, (2) the cost of added technical activities to develop complete requirements and design models, (3) test planning costs, and (4) the cost of all training associated with these activities.

- **Appraisal costs** include activities to gain insight into product condition the “first time through” each process.

Examples of appraisal costs include cost of conducting technical reviews, data collection and metrics evaluation, and testing and debugging.

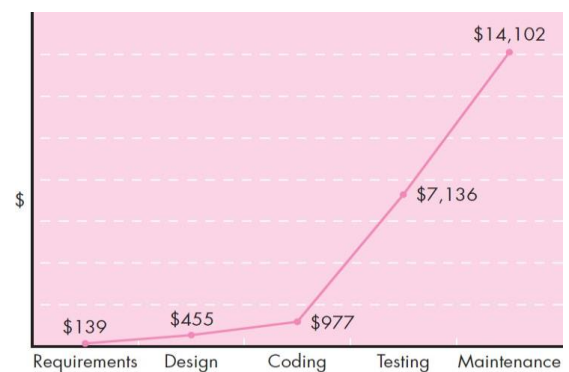


Fig. Relative cost of correcting errors and defects

- **Failure costs** are those that would disappear if no errors appeared before or after shipping a product to customers. Failure costs may be subdivided into internal failure costs and external failure costs.
 - **Internal failure costs** are incurred when you detect an error in a product prior to shipment. Cost required to perform rework (repair) to correct an error is an example of internal failure cost.
 - **External failure costs** are associated with defects found after the product has been shipped to the customer. Examples of external failure costs are complaint resolution,

product return and replacement, help line support, and labor costs associated with warranty work. A poor reputation and the resulting loss of business is another external failure cost.

❖ SOFTWARE QUALITY ASSURANCE

The first formal quality assurance and control function was introduced at Bell Labs in 1916 and spread rapidly throughout the manufacturing world. Quality assurance establishes the infrastructure to build high quality software that supports solid software engineering methods, project management techniques, and quality control actions. In addition, quality assurance consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control actions.

Software Quality Assurance (SQA) is defined as a planned and systematic approach to the evaluation of the quality of software product standards, processes, and procedures. SQA is an umbrella activity that *is applied in each step of software engineering*. SQA encompasses procedures for the effective application of methods and tools, oversight of quality control activities such as technical reviews and software testing, procedures for change management, procedures for assuring compliance to standards, and measurement and reporting mechanisms. The basic elements of SQA are: standards, review and audit, testing, error collection and analysis, change management, education and training, vendor management, security management, safety and risk management.

Before software quality assurance (SQA) activities can be initiated, it is important to define *software quality* at a number of different levels of abstraction. Once you understand what quality is, a software team must identify a set of SQA activities that will filter errors out of work products before they are passed on.

Product evaluation and process monitoring are the **SQA activities** that assure the software development and control processes. Products are monitored for conformance to standards and processes are monitored for conformance to procedures. Software quality assurance is composed of a variety of tasks associated with two different constituencies—the software engineers who do technical work and an

SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting. The SQA group serves as the customer's in-house representative i.e. the people who perform SQA must look at the software from the customer's point of view.

★ Element of SQA

Software quality assurance encompasses a broad range of concerns and activities that focus on the management of software quality. These can be summarized in the following manner:

- **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents, upon following these rules the software will be have better quality.
- **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors.
- **Testing:** Software testing is a quality control function that has one primary goal—to find errors.
- **Error/defect collection and analysis:** SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change management:** Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and confusion almost always leads to poor quality.
- **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders.
- **Vendor management:** Three categories of software are acquired from external software vendors—*shrink-wrapped packages* (e.g., Microsoft Office), a *tailored shell* that provides a basic skeletal structure that is custom tailored to the needs of a purchaser, and *contracted software* that is custom designed and constructed from specifications provided by the customer organization. The job of the SQA organization is to ensure that high-quality software results by suggesting specific quality

practices.

- **Security management:** The software must have quality to control the cybercrime and new government regulations regarding privacy, every software organization should institute policies that protect data at all levels, establish firewall protection for WebApps etc.
- **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
- **Risk management.** Although the analysis and mitigation of risk is the concern of software engineers, the SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

Benefits of Software Quality Assurance (SQA)

- SQA produces high quality software.
- High quality application saves time and cost.
- SQA is beneficial for better reliability.
- SQA is beneficial in the condition of no maintenance for a long time.
- High quality commercial software increase market share of company.
- Improving the process of creating software.
- Improves the quality of the software.
- It cuts maintenance costs. Get the release right the first time, and your company can forget about it and move on to the next big thing. Release a product with chronic issues, and your business bogs down in a costly, time consuming, never-ending cycle of repairs.

Disadvantages of Software Quality Assurance (SQA)

- High cost due to extra tools, training, and skilled staff.
- Time-consuming process that may delay project delivery.
- Administrative overhead from documentation and reporting.
- Resource-intensive, requiring experienced QA personnel.
- Resistance to change from development teams.
- Not foolproof, defects may still remain.
- Complex to manage in large projects.

6.2 SOFTWARE REVIEWS, FORMAL TECHNICAL REVIEW

❖ SOFTWARE REVIEWS

Software review is a systematic inspection of software by one or more individuals who work together to find and resolve errors and defects in the software.

Technical work needs reviewing for the same reason that pencils need erasers. Thus, software reviews are a "filter" for the software engineering process. That is, reviews are applied at various points during software development and serve to uncover errors and defects that can then be removed. Software reviews "purify" the software engineering activities (*analysis, design, and coding*). In general, six steps are employed for software review: planning, preparation, structuring the meeting, noting errors, making corrections (done outside the review), and verifying that corrections have been performed properly. These are the mechanisms that explain the effectiveness and necessity of software reviews .

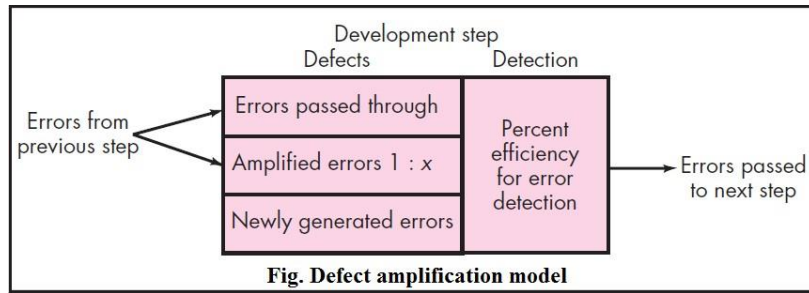
1. Cost impact of software defects

The terms *defect* and *fault* are synonymous within the context of the software process that implies a quality problem discovered *after* the software has been released to end users (or to another framework activity in the software process).

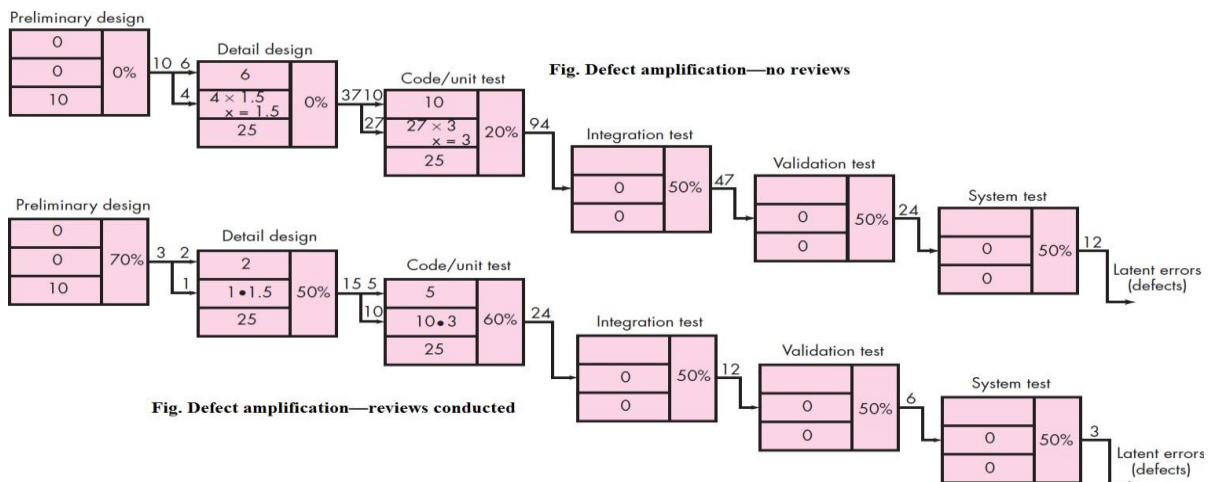
A technical review (TR) is the most effective filter from a quality control standpoint. The primary objective of technical reviews is to find errors during the process so that they do not become defects after release of the software. The obvious benefit of technical reviews is the early discovery of errors so that they do not propagate to the next step in the software process. A number of industry studies indicate that design activities introduce between 50 and 65 percent of all errors during the software process. However, review techniques have been shown to be up to 75 percent effective in uncovering design flaws.

2. Defect Amplification and Removal

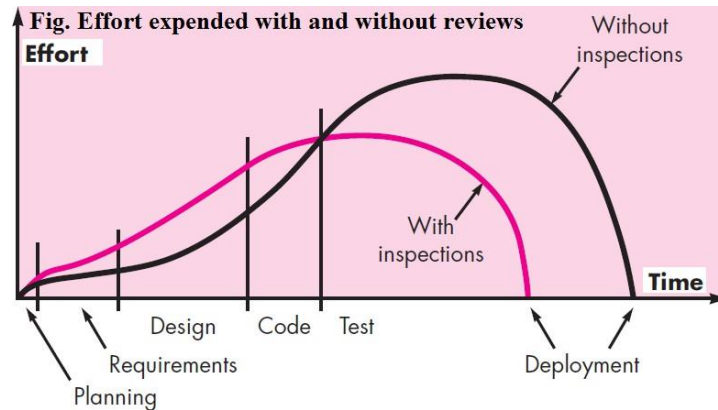
A *defect amplification model* can be used to illustrate the generation and detection of errors during the design and code generation actions of a software process. The model is illustrated schematically in figure.



Review may fail to uncover newly generated errors and errors from previous steps, resulting in some number of errors that are passed through. In some cases, errors passed through from previous steps are amplified (amplification factor, x) by current work. The box subdivisions represent each of these characteristics and the percent of efficiency for detecting errors, a function of the thoroughness of the review. The following figure illustrates a hypothetical example of defect amplification for a software process with no reviews and reviews are conducted.



The effort expended when reviews are used does increase early in the development of a software increment, but this early investment for reviews pays dividends because testing and corrective effort is reduced. As important, the deployment date for development with reviews is sooner than the deployment dates without reviews.



Types of Software Reviews

There are mainly 3 types of software reviews:

1. Software Peer Review

Peer review is the process of assessing the technical content and quality of the product and it is usually conducted by the author of the work product along with some other developers.

Peer review is performed in order to examine or resolve the defects in the software, whose quality is also checked by other members of the team.

Peer Review has following types:

- **Code Review:** Computer source code is examined in a systematic way.
- **Pair Programming:** It is a code review where two developers develop code together at the same platform.
- **Walkthrough:** Members of the development team is guided by author and other interested parties and the participants ask questions and make comments about defects.
- **Technical Review:** A team of highly qualified individuals examines the software product for its client's use and identifies technical defects from specifications and standards.

- **Inspection:** In inspection the reviewers follow a well-defined process to find defects.

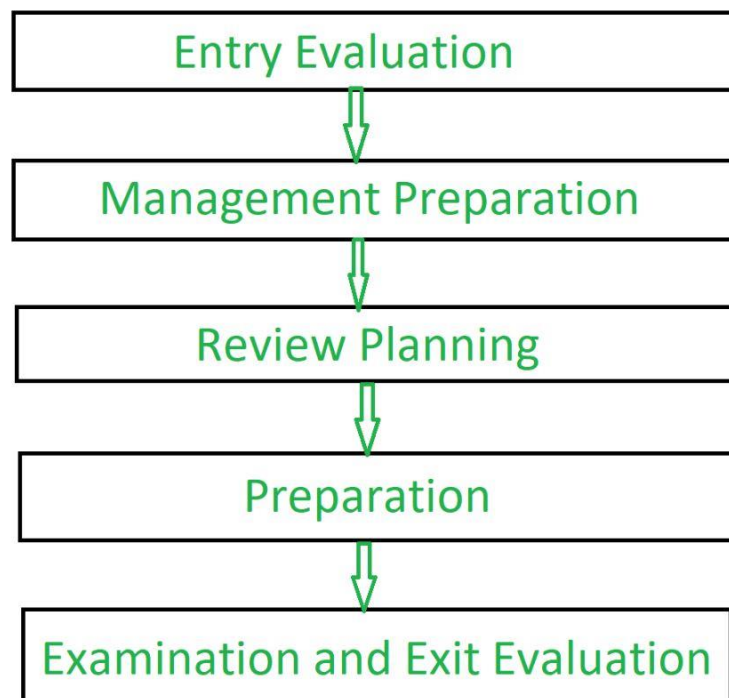
2. Software Management Review

Software Management Review evaluates the work status. In this section decisions regarding downstream activities are taken. A software management review evaluates the project status rather than technical details.

3. Software Audit Review

Software Audit Review is a type of external review in which one or more critics, who are not a part of the development team, organize an independent inspection of the software product and its processes to assess their compliance with stated specifications and standards. This is done by managerial level people.

Process of Software Review:



1. **Entry Evaluation:** By confirming documentation, fulfilling entry requirements and assessing stakeholder and team preparation, you can determine the software's availability.
2. **Management Preparation:** To get ready for the review process, assign roles, gather resources and provide brief management.
3. **Review Planning:** Establish the review's goals and scope, invite relevant parties and set a time for the meeting.
4. **Preparation:** Distribute appropriate resources, give reviewers time to get familiar and promote issue identification to help them prepare.
5. **Examination and Exit Evaluation:** Reviewers should collaborate to examine the results, record concerns, and encourage candid communication in meeting. It checks the results, makes plans to fix reported problems, and evaluates how well the process works.

Importance of Software Reviews:

- Detect defects early in the development cycle
- Reduce cost and time of fixing errors
- Improve overall software quality and reliability
- Enhance security by identifying vulnerabilities
- Ensure adherence to standards and requirements
- Encourage knowledge sharing and team learning
- Increase productivity and reduce rework
- Improve customer satisfaction and trust

❖ FORMAL TECHNICAL REVIEW

A *formal technical review* (FTR) is a software quality control activity performed by software engineers (and others). It is an organized, methodical procedure for assessing and raising the standard of any technical paper, including software objects.

The objectives of an FTR are:

1. To uncover errors in function and logic.

2. To verify that the software under review meets its requirements.
3. To ensure that the software has been represented according to predefined standards.
4. To achieve software that is developed in a uniform manner and
5. To make projects more manageable.

In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and implementation. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen.

The FTR focuses on relatively small portion of a work product. By narrowing the focus, the FTR has a higher likelihood of uncovering errors. Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended.

1. The review meeting

Every review meeting should abide by the following constraints:

- Between three and five people (typically) should be involved in the review.
- Advance preparation should occur but should require no more than two hours of work for each person.
- The duration of the review meeting should be less than two hours.

After the development of work product, the review meeting is attended by the review leader, all reviewers, and the producer. At the end of the review, all attendees of the FTR must decide whether to

- Accept the product without further modification,
- Reject the product due to severe errors (once corrected, another review must be performed), or
- Accept the product conditionally (minor errors and no additional review will be required).

2. Review reporting and record keeping

Record keeping and reporting are other quality assurance activities. It is applied to record all issues that have been raised. At the end of review meeting, a review issue list is produced. A review summary report answers the following questions.

- What was reviewed?

- Who reviewed it?
- What were the findings and conclusions?

3. Review guidelines

The following represents a minimum set of guidelines for formal technical reviews:

1. Review the product, not the producer
2. Set an agenda and maintain it
3. Limit debate and rebuttal
4. Enunciate problem areas but don't attempt to solve every problem noted
5. Take written notes
6. Limit the number of participants and insist upon advance preparation
7. Develop a check list each work product that is likely to be reviewed
8. Allocate resources and time schedule for FTRs.
9. Conduct meaningful training for all reviewers
10. Review your earlier reviews.

6.3 FORMAL APPROACHES TO SQA

Formal approaches to Software Quality Assurance (SQA) in software engineering use rigorous, structured methods like formal logic, mathematical proofs, and disciplined processes (e.g., Six Sigma, cleanroom) to prevent defects, verify correctness, and ensure quality through techniques such as code inspection, formal reviews, static analysis, model checking, and defined testing strategies (unit, integration, performance). These methods focus on process control, data-driven defect analysis (like Pareto analysis), and verifying adherence to standards, moving beyond basic testing to mathematically ensure software meets specifications.

Objectives of Formal SQA Approaches

- To improve software quality in a measurable way.
- To detect and prevent defects early in the software life cycle.
- To ensure compliance with standards and procedures.
- To achieve consistency and reliability in software development.

- To support continuous process improvement.

Components of Formal Approaches to SQA

1. Formal Technical Reviews (FTR)

Formal Technical Review (FTR) is a software quality control activity performed by software engineers. It is an organized, methodical procedure for assessing and raising the standard of any technical paper, including software objects. Finding flaws, making sure standards are followed, and improving the product or document under review's overall quality are the main objectives of a formal technical review (FTR).

Types of FTR:

- Walkthroughs
- Inspections
- Peer reviews

2. Formal Methods

Formal methods in software engineering use mathematical rigor to specify, design, and verify software, ensuring high reliability and safety, especially for critical systems, by moving beyond traditional testing to mathematically prove system correctness through formal logic and models.

Examples: Z notation, Vienna Development Method (VDM), Petri Nets, Finite State Machines (FSM).

3. Statistical Software Quality Assurance

- Uses quantitative data to assess and improve quality
- Involves defect data collection and analysis
- Applies Pareto principle (80% defects caused by 20% problems)
- Helps identify root causes of defects

4. Software Metrics

- Measure software product and process characteristics
- Provide objective data for quality evaluation

Common Metrics:

- Defect density
- Cyclomatic complexity
- Mean Time Between Failures (MTBF)
- Lines of Code (LOC)

5. Software Process Standards

- Define guidelines and best practices for software development
- Ensure consistency, repeatability, and quality assurance

Examples:

- ISO 9001
- CMMI (Capability Maturity Model Integration)
- IEEE Software Engineering Standards

6. Six Sigma for Software Engineering

- Six Sigma is a data-driven approach for software quality improvement.
- It focuses on reducing defects and process variation using statistical techniques.
- The goal of Six Sigma is to achieve **3.4 defects per million opportunities (DPMO)**, indicating very high quality.
- It emphasizes measurement, analysis, and continuous improvement of software processes.

7. Quality Audits

- Formal examination of software processes and products
- Ensures adherence to standards, procedures, and plans

- Identifies gaps and areas for improvement.

6.4 STATISTICAL QUALITY ASSURANCE, SOFTWARE RELIABILITY

❖ STATISTICAL QUALITY ASSURANCE

Statistical quality assurance reflects a growing trend throughout industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the *vital few*).
4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

Six Sigma for Software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today. Originally popularized by Motorola in the 1980s, the Six Sigma strategy “is a rigorous and disciplined methodology that uses data and statistical analysis to measure and improve a company’s operational performance by identifying and eliminating defects’ in manufacturing and service-related processes”. The term Six Sigma is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high quality standard. The Six Sigma methodology defines three core steps:

- Define customer requirements and deliverables and project goals via well-defined methods of customer communication.
- Measure the existing process and its output to determine current quality performance (collect defect metrics).

- Analyze defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps:

- Improve the process by eliminating the root causes of defects.
- Control the process to ensure that future work does not reintroduce the causes of defects.

These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

❖ SOFTWARE RELIABILITY

Software reliability is defined in statistical terms as “the probability of failure-free operation of a computer program in a specified environment for a specified time”. For example: program X is estimated to have a reliability of 0.999 over eight elapsed processing hours.

The reliability of a computer program describes the acceptance limit of its overall quality. Unlike many other quality factors, the software reliability can be measured directly and estimated using historical and development data. The software failure determines the software reliability that may be frequently appeared on the program and can be corrected either frequently or after long time.

Measure of Reliability and Availability

In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. Thus, for computer system, a simple measure of reliability is *meantime-between-failure* (MTBF):

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Where the acronyms MTTF and MTTR are *mean-time-to-failure* and *mean-time-to-repair* respectively

MTBF (Mean Time Between Failures):

The average time a repairable system operates without failing Indicates how stable a system is; a higher MTBF means fewer failures.

MTTF (Mean Time To Failure):

The average lifespan of a *non-repairable* component or system before it needs replacement. Its Measures **reliability** of software before any failure occurs.

MTTR (Mean Time To Repair):

When the software fails, it requires some time to fix the error. MTTR is the measure of the average time system takes to find the errors that cause the failure and to fix them.

Software availability is the probability that a program is operating according to requirements at a given point in time and is defined as:

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

6.5 ISO 9000 QUALITY STANDARD, SQA PLAN

❖ ISO 9000 QUALITY STANDARDS

International Standard Organization (ISO) is a consortium of 63 countries to formulate and foster standardization. It specifies guidelines for maintaining a quality system. The main consideration of ISO is that *“If proper process is followed for production then good quality products are bound to follow automatically.”*

ISO 9000 is a generic name given to a family of standards developed to provide a framework around which a quality management system can effectively be implemented. The ISO 9000 is a series of 3 standards ISO 9001, 9002, and 9003.

- **ISO 9001:** Applies to organizations engaged in *design, development, production, and servicing* of goods, so applicable to most software development organization.
- **ISO 9002:** Applies to organizations which do not design products but are only involved in *production*. E.g. steel and car manufacturing industries (they buy plant and product design from others).
- **ISO 9003:** Applies to organizations involved only in **installation** and

testing of products.

Shortcomings of ISO Certification

- Requires good software production process to adhere to high quality but no guidelines for defining an appropriate process.
- Variations in the norms of awarding certificates among different ISO agencies in different country and places.
- In manufacturing industry there exists a link between process quality and product quality. Good process quality product but software is not manufactured
i.e. it is developed according as the end user requirements.

❖ SQA PLAN

A Software Quality Assurance (SQA) Plan in software engineering is a detailed document outlining the processes, standards, tools, and responsibilities to ensure software meets quality requirements throughout the development lifecycle, focusing on preventing defects, not just finding them, by defining reviews, testing, code control, and corrective actions for consistent quality and customer satisfaction.

An SQA plan will include several components, such as purpose, references, configuration and management, tools, code controls, testing methodology, problem reporting and remedial measures, and more, for easy documentation and referencing.

An SQA plan serves three purposes. It includes the following:

- Determining the QA duties assigned to the concerned team.
- A list of the areas that require review, audit, and examination.
- Determines the work products for SQA.

SQA plan:



Importance of Software Quality Assurance Plan

- **Quality Standards and Guidelines:** The SQA Plan lays out the requirements and guidelines to make sure the programme satisfies predetermined standards for quality.
- **Risk management:** It is the process of recognizing, evaluating and controlling risks in order to reduce the possibility of errors and other problems with quality.
- **Standardization and Consistency:** The strategy guarantees consistent methods, processes, and procedures, fostering a unified and well-structured approach to quality assurance.
- **Customer Satisfaction:** The SQA Plan helps to ensure that the finished product satisfies customer needs, which in turn increases overall customer satisfaction.

- **Resource optimization:** It is the process of defining roles, responsibilities, and procedures in order to maximize resource utilization and minimize needless rework.
- **Early Issue Detection:** SQA Plans help identify problems early on, which lowers the expense and work involved in fixing them.

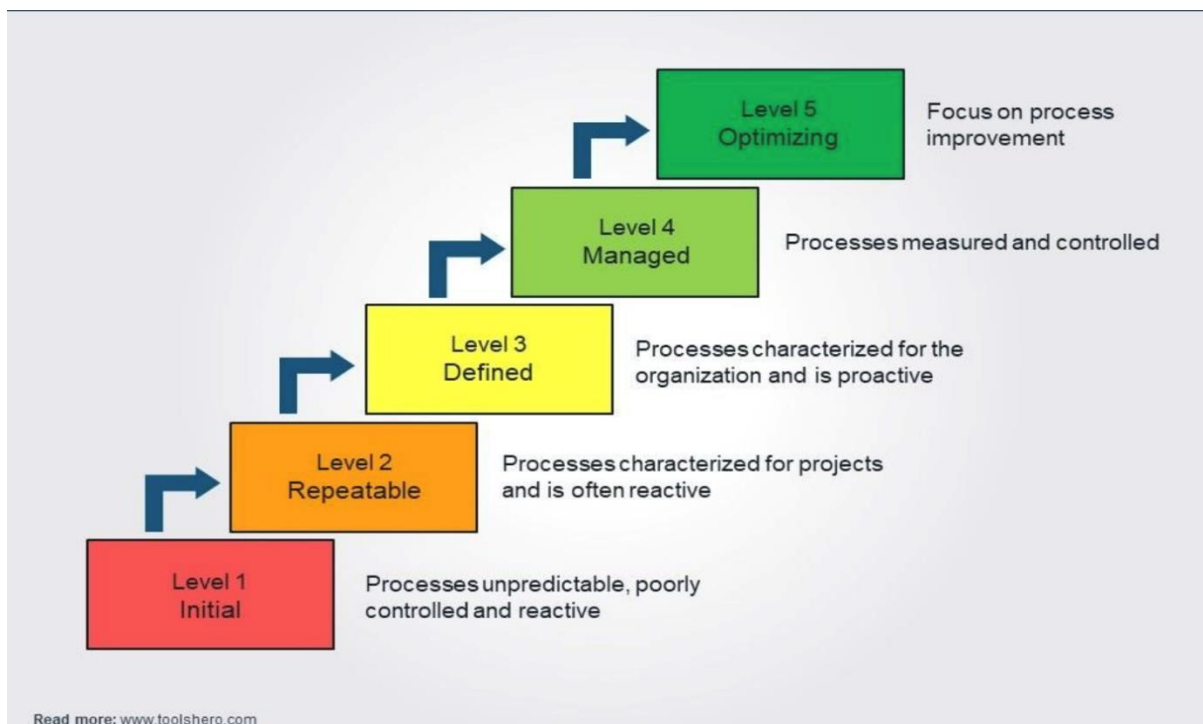
6.6 CAPABILITY MATURITY MODEL(CMM)

The Capability Maturity Model (CMM) is a tool used to improve and refine software development processes. It provides a structured way for organizations to assess their current practices and identify areas for improvement.

Capability Maturity Model (CMM) was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987. It is not a software process model. It is a framework that is used to analyze the approach and techniques followed by any organization to develop software products. It also provides guidelines to enhance further the maturity of the process used to develop those software products.

It is based on profound feedback and development practices adopted by the most successful organizations worldwide. This model describes a strategy for software process improvement that should be followed by moving through 5 different levels.

Levels of Capability Maturity Model (CMM)



Level-1: Initial (Ad hoc / Chaotic)

- No KPIs defined.
- Processes followed are Adhoc and immature and are not well defined.
- Limited project management capabilities, such as no systematic tracking of schedules, budgets, or progress.
- We have limited communication and coordination among team members and stakeholders.
- No formal training or orientation for new team members.
- Little or no use of software development tools or automation.
- Highly dependent on individual skills and knowledge rather than standardized processes.

- High risk of project failure or delays due to a lack of process control and stability.
Eg: Student studies randomly the night before exams. Sometimes passes, sometimes fails.

Level-2: Repeatable

- Focuses on establishing basic project management policies.
- Experience with earlier projects is used for managing new similar-natured projects.
- Basic project planning & tracking.
- Past successes can be repeated.
- Requirements & deliverables documented.
- Simple configuration & change control.
- Some training provided.
- Still limited standardization.
Eg: Student starts using old question papers and routines. Can repeat success in similar exams.

Level-3: Defined

- At this level, documentation of the standard guidelines and procedures takes place.
- Organization-wide standards established.
- Processes well-documented & consistent.
- Training and orientation formalized.
- Defined roles & responsibilities.
- Better communication across teams.
- Improved predictability & quality.
Eg: Student follows a proper timetable, notes, and study techniques. Consistent preparation.

Level-4: Managed

- At this stage, quantitative quality goals are set for the organization for software products as well as software processes.
- The measurements made help the organization to predict the product and process quality within some limits defined quantitatively.
- **Software Quality Management:** It includes the establishment of plans and strategies to develop quantitative analysis and understanding of the product's quality.
- **Quantitative Management:** It focuses on controlling the project performance quantitatively.
Eg: Student tracks marks, weak areas, and studies based on data. Performance becomes predictable.

Level-5: Optimizing

- This is the highest level of process maturity in CMM and focuses on continuous process improvement in the organization using quantitative feedback.
- The use of new tools, techniques, and evaluation of software processes is done to prevent the recurrence of known defects.
- **Process Change Management:** Its focus is on the continuous improvement of the organization's software processes to improve productivity, quality, and cycle time for the software product.
- **Technology Change Management:** It consists of the identification and use of new technologies to improve product quality and decrease product development time.
- **Defect Prevention** It focuses on the identification of causes of defects and prevents them from recurring in future projects by improving project-defined processes.
Eg: Student keeps improving study methods, tries new techniques, and becomes a topper.

Importance of Capability Maturity Model

- **Optimization of Resources:** CMM helps businesses make the best use of all of their resources, including money, labor, and time. Organizations can improve the effectiveness of resource allocation by recognizing and getting rid of unproductive practices.

- **Comparing and Evaluating:** A formal framework for benchmarking and self-evaluation is offered by CMM. Businesses can assess their maturity levels, pinpoint their advantages and disadvantages, and compare their performance to industry best practices.
- **Management of Quality:** CMM emphasizes quality management heavily. The framework helps businesses apply best practices for quality assurance and control, which raises the quality of their goods and services.
- **Enhancement of Process:** CMM gives businesses a methodical approach to evaluate and enhance their operations. It provides a road map for gradually improving processes, which raises productivity and usefulness.
- **Increased Output:** CMM seeks to boost productivity by simplifying and optimizing processes. Organizations can increase output and efficiency without compromising quality as they go through the CMM levels.

Limitations of Capability Maturity Model (CMM)

- **Focuses mainly on process, not product**
CMM improves processes but does not directly guarantee high-quality software products.
- **Costly to implement**
Requires significant investment in training, tools, documentation, and audits.
- **Time-consuming**
Achieving higher maturity levels may take several years.
- **Documentation heavy**
Emphasizes formal documentation, which can slow development.
- **Not suitable for small organizations**
Small teams may find CMM too complex and resource-intensive.
- **Does not ensure defect-free software**
Even at higher levels, defects can still exist.
- **Rigid and less flexible**
Difficult to adapt quickly to changing requirements or agile environments.

